

Introduction to Lean 4

Natural Numbers

MATH230

School of Mathematics and Statistics
University of Canterbury

- ① Formal Mathematics in Lean
- ② Natural Numbers: Arithmetic
- ③ Natural Numbers: Induction
- ④ Tactic: rEwRITE
- ⑤ Arithmetic Predicates

If we want extend Curry-Howard to mathematics, then we need a type system that allows us to express the mathematical objects we're interested in. For us, this will mean $\mathbb{N}$ the natural numbers and their arithmetic.

Type of natural numbers	Inductive types
Addition	functions on inductive types
Multiplication	functions on inductive types

Inductive types are enough to express a lot of basic mathematical (and logical) constructions. However, more would be required of the type system to encapsulate all of mathematics.

Type of Natural Numbers

Defining the type of natural numbers $\mathbb{N}$ is done similarly to the definition by Peano Arithmetic. We say zero is a natural number, and then we have a function which generates new natural numbers from old.

```
inductive N where
  | zero : N
  | succ : N -> N
```

As part of this definition we get the first two axioms of PA; that no successor is equal to zero and that the successor function is injective. These are still theorems in Lean, but their proof is straightforward - simply stating, they are so “by definition”.

Addition by Pattern Matching

Again, the arithmetic operations are defined in the same manner as Peano Arithmetic. For example, we use pattern matching on the second argument to define addition:

```
def natadd (m n : N) : N :=  
  match n with  
  | zero    => m  
  | succ k => succ (add m k)  
  
instance : Add N where  
  add := natadd
```

Pattern matching works on the idea that the second input can only be in one of two forms: either zero, or the successor of k some other natural number. Therefore, in order to define addition it is sufficient to define it in both of these cases.

Multiplication by Pattern Matching

Similarly, we use pattern matching on the second argument to define multiplication:

```
def natmul (m n : N) : N :=  
  match n with  
  | zero      => zero  
  | succ k    => (mul m k) + m  
  
instance : Mul N where  
  mul := natmul
```

Pattern matching works on the idea that the second input can only be in one of two forms: either zero, or the successor of k some other natural number. Therefore, in order to define multiplication it is sufficient to define it in both of these cases.

Axioms PA3 - PA6 now follow by definition. That is to say, they simply state the properties we have defined addition and multiplication with.

```
def PA3 (m : N) : m + zero = m
      := by rfl
def PA4 (m n : N) : m + (succ n) = succ (m + n)
      := by rfl
def PA5 (m : N) : m * zero = zero
      := by rfl
def PA6 (m n : N) : m * (succ n) = (m * n) + m
      := by rfl
```

Lean can verify these equations by using the definitions of $+$ and $*$ given above. We use the tactic “rfl” which instructs Lean to perform the computation and then verify that the two sides of the equation are indeed $=$ by definition.

Mere Computation

Each of the theorems on the previous page - axioms of Peano arithmetic - hardly needed any proving. This is because they are simply stating the definitions of addition and multiplication.

In this way it is said that each of them are true up to computation. They are “merely computation”. If both sides of the equation can be normalised (computed) to the same value, then Lean can use computation to close the goal.

Any proof which is in a state that can be closed by mere computation can be closed with “rfl” tactic.

Anything more interesting will not be mere computation. Recall we used induction to prove many of the basic properties of addition and multiplication. The code below shows the basic induction pattern:

```
theorem zero_add :  $\forall x : \mathbb{N}, \text{zero} + x = x :=$   
  by  
    intro a  
    induction a with  
    | zero          => _  
    | succ k ih => _
```

This is saying we are to do induction on a . This means we need to consider two cases (i) $a = \text{zero}$, and (ii) $a = \text{succ } k$. In the second case we assume we have a proof of the theorem at k .

$$\text{ih} : \text{zero} + k = k$$

Induction: Base Case

Often the base cases are mere computation and can therefore can be closed by `rfl`.

```
theorem zero_add :  $\forall x : \mathbb{N}, \text{zero} + x = x :=$   
  by  
    intro a  
    induction a with  
    | zero      => rfl  
    | succ k ih => _
```

In this theorem the base case is asking us to prove

$$\text{zero} + \text{zero} = \text{zero}$$

We can compute the value of `zero + zero` and find that both sides are equal.

Induction: Induction Step

In this case the induction step is the following:

$$\text{ih} : \text{zero} + k = k \vdash \text{zero} + \text{succ } k = \text{succ } k$$

We can show this to be the case with PA4 and the induction hypothesis.

$$\begin{aligned} \text{zero} + \text{succ } k &= \text{succ } (\text{zero} + k) && \text{by [PA4]} \\ &= \text{succ } k && \text{by [ih]} \end{aligned}$$

We can write the proof in this style with the “calc” tactic.

```
theorem zero_add :  $\forall x : \mathbb{N}, \text{zero} + x = x :=$   
  by  
    intro a  
    induction a with  
    | zero      => rfl  
    | succ k ih =>  
      calc  
        zero + succ k = succ (zero + k) := by rw [PA4]  
        _ = succ k           := by rw [ih]
```

This makes for a very clear and readable proof.

We can write a more compact proof using the “`rw`” tactic.

```
theorem zero_add :  $\forall x : \mathbb{N}, \text{zero} + x = x :=$   
  by  
    intro a  
    induction a with  
    | zero      => rfl  
    | succ k ih => rw [PA4, ih]
```

This makes for a more succinct proof.

Used in this way `rw` works on the goal by recognising a pattern in a theorem and fitting it to the goal. It collects the $\forall$ elimination and $=E$ steps we made on paper into one neat step.

We apply rewrites to the goal until it is in a form that can be closed by the `rfl` tactic.

Let's focus on the proofstate before and after the first `rw`:

```
1 goal
k : N
ih : zero + k = k
⊢ zero + k.succ = k.succ
```

Using the tactic `rw[PA4]` updates the goal of the proofstate.

```
1 goal
k : N
ih : zero + k = k
⊢ (zero + k).succ = k.succ
```

Recall the statement of PA4.

```
def PA4 (m n : N) : m + (succ n) = succ (m + n) := by rfl
```

`rw` instantiates (chooses `m n`) PA4 in such away that it can match the RHS of PA4 with a term in the goal.

In general `rw` uses an equational theorem $a = b$. It rewrites all instances of a with b in the goal.

Example:

```
1 goal
k : N
ih : k * zero.succ = k
⊢ k.succ * zero.succ = k.succ
```

At this point we notice the pattern: `k.succ * zero.succ`.

This appears in the left hand side of PA6.

```
def PA6 (m n : N) : m * (succ n) = (m * n) + m := by rfl
```

When we instantiate $m = k.succ$ and $n = zero$.

For this reason `rw[PA6]` will make progress on this goal.

Specifying `rw` Pattern

If there are multiple instances of a pattern in the goal, then you will need to specify which instance of the theorem you want to use in the rewrite. This can be done by specifying the terms to instantiate the theorem at. As we did on paper, we must be explicit about the choice of terms made when doing $\forall$ elimination.

Example: Multiple Additions

```
1 goal
a n : ℕ
ih : a.succ * n = a * n + n
⊢ (a * n + (n + a)).succ = (a * n + a + n).succ
```

`rw [add_comm n a]` will commute the second addition $(n + a)$.

`rw [add_comm]` will commute the addition $(a * n + (n + a).succ)$

Backwards rewrites

Lean uses `rw` by reading an equation theorem-name: $a = b$ and replacing a with b in the goal.

If one wishes to use the equation $a = b$ to replace b with a in the goal, then one can write `rw[<-theorem-name]`. That is, the use of arrow “<-” tells Lean to read the equation right-to-left in the rewrite.

```
1 goal
a b c u v : N
hu : b = a + u
hv : c = b + v
⊢ c = a + u + v
```

In order to close this goal, we need to sub b for $a + u$.

Backwards Rewrites

Consider the following proof state from before:

```
1 goal
a n : ℕ
ih : a.succ * n = a * n + n
⊢ (a * n + (a + n)).succ = (a * n + a + n).succ
```

It looks like the two are equal. However, the parentheses don't match. We need to use `add_assoc` to rebracket:

```
def add_assoc : ∀ x y z : ℕ,
  (x + y) + z = x + (y + z) := by ...
```

There are two paths to close the goal from here:

`rw [add_assoc]` will rebracket $(a * n + a) + n$

`rw [<-add_assoc]` will rebracket $a * n + (a + n)$

Arithmetic Predicates

On top of the arithmetic of the natural numbers we can begin to prove things about higher and higher properties.

Including the relations: $<, \leq, |, \dots$

These three are statements about the existence of solutions to certain equations.

```
def LT (m n : N) :=  $\exists$  k : N, n = m + (succ k)
def LTE (m n : N) :=  $\exists$  k : N, n = m + k
def DIV (m n : N) := ( $\exists$  k : N, n = m*k)  $\wedge$  (m  $\neq$  zero)
```

Recall that terms of the type $\exists x : \mathbb{N}, Px$ are pairs (k, t) where $k : \mathbb{N}$ and $t : Pt$. In order to make use of hypotheses like $a \leq b$ requires us to be able to access the elements of these pairs.

Proving them will require `Exists.intro` `__` `__`

Tactic: obtain

```
1 goal
a b : N
t1 : a < b
⊢ a.succ < b.succ
```

To make use of the hypothesis t_1 we need to access the terms in the underlying pair

```
obtain ⟨k,pk⟩ := t1
```

This introduces the natural number k and a proof that k satisfies the equation defining the relation: k is the witness to $a < b$.

```
1 goal
a b k : N
pk : b = a + k.succ
⊢ a.succ < b.succ
```

Exists Introduction

```
1 goal
a b k : N
pk : b = a + k.succ
⊢ a.succ < b.succ
```

We need to provide an x such that $\text{succ } b = \text{succ } a + \text{succ } x$. This same $x = k$ is the natural number we are after!

```
apply Exists.intro k
```

```
1 goal
case intro
a b k : N
pk : b = a + k.succ
⊢ b.succ = a.succ + k.succ
```

Two rewrites will close this goal; `succ_add` and `pk`.

Further Reading

These slides were prepared with the aid of the following references.
These should be consulted for further detail on the topics.

Type Theory and Functional Programming, Simon Thompson

Theorem Proving in Lean 4

Each of these are freely available on the world wide web.